

Anvil · P-01 · CRDT-Native OLTP Engine

Team: TechGenDM | **Track:** P-01 · Conflict-Free Collaborative OLTP | **Repository:** github.com/TechGenDM/Anvil-Conflict-Free-Collaborative-OLTP | **FK Policy:** Tombstone (declared uniform)

We present a pure-Python, coordinator-free CRDT engine that supports concurrent INSERT, UPDATE, and DELETE across N peers with no central authority. The engine implements per-column Multi-Value Registers (MVR) over vector clocks, an escrow-based uniqueness protocol, a tombstone FK policy, and a bidirectional merge-sync that is commutative, associative, and idempotent. All metadata grows $O(\text{writers})$, not $O(\text{writes})$.

1 · Lattice Choices Per Data Type

1.1 Row Membership — Observed-Remove Set (OR-Set) via Tombstone Flag

Each row carries a `tombstone: bool` flag and a `tombstone_clock: VectorClock`. Rows are **never physically deleted** from the in-memory store. A DELETE operation sets `tombstone = True` and records the deleting peer's vector clock at that moment. During merge, tombstone is **monotonically propagated**: `merged.tombstone = self.tombstone OR other.tombstone`. This is the standard OR-Set "add-wins" / "remove-wins" choice — we choose **remove-wins** (tombstone wins) because it is the safe default for OLTP: a confirmed delete must not be resurrected by a concurrent insert arriving late.

The tombstone clock is merged via component-wise maximum, preserving causal history. Tombstoned rows are excluded from `snapshot_state()` but retained in the internal store for sync correctness.

1.2 Cell Values — Multi-Value Register (MVR) over Vector Clocks

Each column value is stored as a `CRDTCell(value, clock, conflicts[])`. The merge rule is:

Relationship between clocks	Merge outcome
<code>self.clock</code> dominates <code>other.clock</code>	Keep <code>self</code> — causally newer
<code>other.clock</code> dominates <code>self.clock</code>	Keep <code>other</code> — causally newer
Neither dominates (concurrent)	Multi-Value Register: retain both in <code>conflicts[]</code> , merge clocks via component-wise max

When a single winner value is required (e.g., for `snapshot_state()`), `winner_value()` sorts all concurrent values lexicographically by their string representation and returns the smallest. This is deterministic, peer-independent, and requires no coordination.

Why MVR over LWW? Last-Write-Wins requires a globally agreed-upon wall-clock or Lamport timestamp, which is unavailable in a coordinator-free setting. MVR preserves all concurrent writes and resolves them deterministically — the bench's `cell-level-strict` scenario explicitly tests that a row updated concurrently on two columns retains both updates, which LWW-on-row physically fails.

1.3 Vector Clocks — Bounded Metadata

Each `VectorClock` is a dict `{peer_id: int}`. Entries are per-writer, not per-write: a peer's counter increments once per local operation, not once per cell. After every sync, `prune(active_peers)` removes entries for peers no longer in the known set. This bounds metadata growth to **$O(\text{number of writers})$** , not $O(\text{number of writes})$ — satisfying the problem statement's metadata-growth constraint.

1.4 Scoring Axes Addressed by Lattice Choices

Bench Axis	Weight	Lattice mechanism
cell-level:u1 / cell-level-strict	10%	MVR per column — concurrent updates to different columns both survive
convergence	30%	Merge is commutative + associative → eventual consistency
order-invariance (chaos)	10%	Merge commutativity → same final state regardless of sync order
randomized	15%	Property-based invariants hold for any seed — no hardcoding possible

2 · Uniqueness Protocol

2.1 Problem Statement

Standard CRDT row membership does not enforce UNIQUE constraints: two peers can independently INSERT rows with the same email, and after sync both rows exist — violating the constraint. A coordinator-free uniqueness protocol must resolve conflicts deterministically without communication at write time.

2.2 Escrow Log Design

We implement an **EscrowLog** — a CRDT map from `(table, cols_tuple, vals_tuple)` to a list of `(peer_id, row_pk)` claimants. The escrow log is itself a CRDT: its merge operation is a set union of claimants (idempotent, commutative, associative).

On every INSERT, the executor calls `escrow.claim(table, cols, vals, peer_id, row_pk)` to register the claim. During sync, both peers' escrow logs are merged bidirectionally before resolution runs.

2.3 Resolution Rule

After escrow merge, `resolve_all(store)` iterates over every `(table, cols, vals)` group:

- **Single claimant:** mark that row `unique_status = "committed"`.
- **Multiple claimants:** `winner = min(claimants, key=(peer_id, row_pk))` lexicographically. Winner → `"committed"`. All others → `"rejected"`.

Rejected rows are retained in the store (for sync correctness) but excluded from `snapshot_state()`. This means the bench's uniqueness assertion — no duplicate emails in live rows — is always satisfied after sync.

2.4 Safety-Net: Post-Sync Full-Table Scan

UPDATE operations can introduce new uniqueness violations (e.g., a peer updates a row's email to match another row's email). The escrow log only captures INSERT-time claims. To handle UPDATE-driven collisions, `sync()` runs a secondary `_resolve_unique_duplicates(store)` pass after escrow resolution: it scans all live rows per unique column, detects duplicates, and marks the loser (by `min(row_pk)`) as `"rejected"`. This covers the bench's `composite_uniqueness` and `high_density` scenarios.

2.5 Composite Uniqueness

The schema registry stores `composite_unique: list[list[str]]` alongside `unique_cols: list[str]`. The escrow log key uses a `tuple(cols)` so composite constraints (e.g., `UNIQUE(user_id, team_id)`) are handled identically to single-column constraints — no special-casing required.

2.6 Idempotency Guarantee

Resolution is deterministic: given the same escrow log and store state, `resolve_all()` always produces the same `unique_status` assignments. Running sync twice produces identical hashes — satisfying the bench's explicit idempotent-sync invariant.

3 · FK Protocol — Tombstone Policy

3.1 Policy Declaration

We declare **tombstone** as our uniform FK-under-partition policy. This means: when a parent row is deleted (tombstoned), child rows referencing it **survive** in the store and remain visible in `snapshot_state()`. Their `fk_status` is set to `"orphaned"`, and their FK column value is preserved (not nullified). This is the correct

choice for the reference scenario: `o1` must appear in `snapshot_state()` even though its parent `u1` is tombstoned.

3.2 FKEnforcer — Three Entry Points

Method	When called	What it does
<code>on_parent_delete(store, table, pk)</code>	On DELETE in executor	Scans all child tables for rows referencing <code>pk</code> ; sets <code>fk_status = "orphaned"</code>
<code>on_child_insert(store, table, row_dict)</code>	On INSERT in executor	Checks if referenced parent is already tombstoned; if so, marks child orphaned immediately
<code>recheck_all(store)</code>	After every sync	Fixed-point iteration: re-validates all FK relationships to handle out-of-order arrivals

3.3 Multi-Level FK Propagation

`recheck_all()` uses a **fixed-point loop**: it iterates until no new orphaned rows are found. This correctly handles multi-level FK chains (e.g., `organizations` → `users` → `orders`): if an organization is tombstoned, the loop propagates `fk_status = "orphaned"` down through users and then orders in successive iterations. The bench's `multi_level_fk` scenario is handled by this mechanism.

3.4 Visibility Rule

`CRDTRow.is_visible(fk_policy)` returns `True` for orphaned rows under tombstone policy (they appear in `snapshot_state()`). Under cascade policy, orphaned rows return `False` (hidden). The policy is set once at engine construction and applied uniformly — switching mid-run is disqualifying per the bench rules.

4 · Sync Protocol

4.1 Algorithm

`sync(peer_a, peer_b)` is a **full-state bidirectional exchange**. It executes the following steps atomically (no partial sync):

```
def sync(peer_a, peer_b):
    # Step 1: Merge known_peers (union)
    combined = peer_a.known_peers | peer_b.known_peers
    peer_a.known_peers = peer_b.known_peers = combined

    # Step 2: Snapshot both stores BEFORE merging (avoid mid-sync contamination)
    a_snap = {t: dict(peer_a.store.get_all_rows(t)) for t in all_tables}
    b_snap = {t: dict(peer_b.store.get_all_rows(t)) for t in all_tables}

    # Step 3: Exchange all rows bidirectionally via CRDTRow.merge()
    for table, rows in a_snap.items():
        for pk, row in rows.items():
            peer_b.store.merge_row(table, pk, row)    # A → B
    for table, rows in b_snap.items():
        for pk, row in rows.items():
            peer_a.store.merge_row(table, pk, row)    # B → A

    # Step 4: Merge escrow logs (set union, idempotent)
    peer_a.escrow.merge(peer_b.escrow)
    peer_b.escrow.merge(peer_a.escrow)

    # Step 5: Resolve uniqueness on both peers
    peer_a.escrow.resolve_all(peer_a.store)
    peer_b.escrow.resolve_all(peer_b.store)
    _resolve_unique_duplicates(peer_a.store)    # UPDATE-driven safety net
    _resolve_unique_duplicates(peer_b.store)

    # Step 6: Recheck FK on both peers (fixed-point)
    FKEnforcer().recheck_all(peer_a.store)
    FKEnforcer().recheck_all(peer_b.store)

    # Step 7: GC – prune vector clocks to known_peers
    for store in (peer_a.store, peer_b.store):
        for row in all_rows(store):
            for cell in row.cells.values():
                cell.clock.prune(active_peers)
            if row.tombstone_clock:
                row.tombstone_clock.prune(active_peers)
```

4.2 Why Snapshot Before Merging

Step 2 snapshots both stores before any merging begins. This prevents a row merged from A into B from being immediately re-read as B's own state and re-merged back into A in the same sync call — which would cause non-deterministic behavior. The snapshot ensures each peer contributes exactly its pre-sync state to the exchange.

4.3 Data Exchanged Per Sync

The current implementation exchanges **all rows** (full-state sync). This is correct and simple. For production, a delta-sync optimization would exchange only rows where the sender's version vector is not dominated by the receiver's — reducing bandwidth to $O(\text{diverged rows})$. The bench does not penalize full-state sync.

5 • Metadata Growth Analysis

Metadata item	Growth bound	Mechanism
Vector clock entries per cell	$O(W)$ — W = number of distinct writers of that cell	<code>prune(active_peers)</code> removes entries for departed peers after every sync
Tombstone clock entries	$O(W)$ — W = peers that have seen the delete	Same prune mechanism applied to <code>tombstone_clock</code>
Escrow log entries	$O(U \times P)$ — U = unique values ever claimed, P = peers	Escrow entries are never removed (needed for correctness); bounded by number of unique constraint violations
Conflict list per cell	$O(C)$ — C = number of concurrent writers of that cell	Bounded by number of peers; in practice 1–3 entries per cell
Tombstoned rows	$O(D)$ — D = total deletes ever	Tombstones are permanent; GC requires distributed consensus (out of scope)

The critical bound is vector clock size: **$O(\text{writers per cell})$** , not $O(\text{writes})$. A cell updated 10,000 times by 3 peers has a vector clock of size 3, not 10,000. This satisfies the problem statement's requirement that metadata growth be bounded by the number of writers, not the number of operations.

6 • Convergence Proof (Informal)

Claim: For any two peers A and B that have observed the same set of operations (possibly in different orders), `snapshot_hash(A) == snapshot_hash(B)` after sync.

Proof sketch:

- **CRDTCell.merge is commutative:** `merge(a, b) == merge(b, a)` — the dominates/concurrent check is symmetric; the conflict list union is a set operation.
- **CRDTCell.merge is associative:** `merge(merge(a, b), c) == merge(a, merge(b, c))` — component-wise max of vector clocks is associative; conflict list union is associative.
- **CRDTRow.merge is commutative and associative:** follows from cell-level commutativity + associativity, plus tombstone monotonicity (boolean OR is commutative and associative).
- **Uniqueness resolution is deterministic:** given the same escrow log (which is a CRDT set — commutative, associative), `resolve_all()` always produces the same `unique_status` assignments. Winner selection by `min(peer_id, row_pk)` is a total order — no ties.
- **FK recheck is deterministic:** `recheck_all()` is a fixed-point computation over a monotone function (orphaned status only grows, never shrinks). Fixed-point of a monotone function on a finite lattice is unique.
- **snapshot_state is deterministic:** rows sorted by PK, columns sorted by name, `json.dumps(sort_keys=True)` — same state always produces same JSON string and same SHA-256 hash.

Therefore: after any sequence of syncs that forms a connected graph over all peers, all peers converge to the same `snapshot_hash()`. The order of sync calls does not affect the final state (order-invariance / chaos scenario). ■

7 • Submission Checklist

Requirement	Status	Location
-------------	--------	----------

Git repository (public)	✓	github.com/TechGenDM/Anvil-Conflict-Free-Collaborative-OLTP
README quickstart (<5 min, clean machine)	✓	Anvil-P-E/bench-p01-crdt/README.md
Reproducibility (Dockerfile / requirements.txt)	✓	bench-p01-crdt/requirements.txt
Demo video (5 min, L3 banner visible)	✓	Submitted separately
3-page writeup PDF	✓	This document
L3 JSON output pasted in submission form	✓	report_p01_final.json
FK policy declared uniform	✓ tombstone	Engine constructor + README
Bench frozen (no modifications to harness)	✓	Only adapters/ourteam.py + engine/ modified

Anvil Hackathon · P-01 · CRDT-Native OLTP Engine · Team TechGenDM · 2026 | Engine: pure Python 3.11, stdlib + sqlglot. No GPU. No network access required. CPU only.